

LLM Math

Wout Vossen

July 19, 2026

1 Notation and Assumptions

Symbol	Description
B	Batch size
T	Context length of passed input
T_q	Context length of the query tensor
T_k	Context length of the key tensor
C	Model hidden dim (embedding dim)
H	Number of attention heads
H_d	Hidden dim of attention heads, where $H_d = C/H$
FF_C	Hidden dim of feed forward layers
L	Number of decoder layers
V	Vocabulary size

Table 1: Notation used through the analysis.

- FLOPs: Total floating point operations
- Calculations are based on decoder as described in "Attention is all you need"
- During prefill, the query and key sequence lengths both equal the input context length, so $T_q = T_k = T$.
- A multiply-accumulate is approximated as 2 FLOPs (one multiplication and one addition).
- I don't count softmaxes, ReLU's, layer norms, reshapings, operations with constants, concats, applications of attention masks as those are trivial compared to the matmuls that happen in the decoder layer.

2 Prefill / forward pass

Let's go through all the major operations of the forward pass during prefill in order of appearance. This would also correspond to the FLOPs necessary for a decode pass without a KV cache.

2.1 Embedding / Positional Encodings

Embeddings are simple lookups and do no floating point operations. We do add a positional encoding to each of the hidden dimensions, which is why we do have FLOPs. These are however trivial compared to the decoder layer.

$$\text{Input shape} = \boxed{B \times T} \quad (1)$$

$$\text{Output shape} = \boxed{B \times T \times C} \quad (2)$$

$$\text{FLOPs} = \boxed{BTC} \quad (3)$$

2.2 Decoder: Q, K, V projections

For each decoder layer L we do a multiplication with Q, K, V matrices. With dimensions $C \times C$ with a $C/H = H_d$ of adjacent columns representing each head.

$$\text{Input shape} = \boxed{B \times T \times C}, \quad (4)$$

$$\text{Output shape} = \boxed{B \times H \times T \times H_d}, \quad (5)$$

$$\text{FLOPs} = L \times 3 \times 2 \times B \times T \times C \times C \quad (6)$$

$$= \boxed{6LBT C^2}. \quad (7)$$

2.3 Decoder: Attention $QK^\top$

For each decoder layer L we do a multiplication of the query and transpose of key projection.

$$\text{Input shape} = \boxed{B \times H \times T_q \times H_d}, \quad (8)$$

$$\text{Output shape} = \boxed{B \times H \times T_q \times T_k}, \quad (9)$$

$$\text{FLOPs} = L \times 2 \times B \times H \times T_q \times H_d \times T_k \quad (10)$$

$$= \boxed{2LBCT^2}. \quad (11)$$

2.4 Decoder: Attention $\text{softmax}(QK^T)V$

For each decoder layer L we multiply the output of the query-key multiplication with the value projection.

$$\text{Input shape} = \boxed{B \times H \times T_q \times T_k}, \quad (12)$$

$$\text{Output shape} = \boxed{B \times H \times T_q \times H_d}, \quad (13)$$

$$\text{FLOPs} = L \times 2 \times B \times H \times T_q \times T_k \times H_d \quad (14)$$

$$= \boxed{2LBCT^2}. \quad (15)$$

2.5 Decoder: Linear attention projection

For each decoder layer a multiplication with a linear layer is done on the concatenation of the multi-head attention.

$$\text{Input shape} = \boxed{B \times T \times C}, \quad (16)$$

$$\text{Output shape} = \boxed{B \times T \times C}, \quad (17)$$

$$\text{FLOPs} = L \times 2 \times B \times T \times C \times C \quad (18)$$

$$= \boxed{2LBT C^2}. \quad (19)$$

2.6 Decoder: Feed forward

In each decoder layer we put the output of attention through a two-layer feed-forward network which first projects from C to FF_C and then back from FF_C to C .

$$\text{Input shape} = \boxed{B \times T \times C}, \quad (20)$$

$$\text{Output shape} = \boxed{B \times T \times C}, \quad (21)$$

$$\text{FLOPs} = L(2 \times B \times T \times C \times \text{FF}_C + 2 \times B \times T \times \text{FF}_C \times C) \quad (22)$$

$$= \boxed{4LBT C \text{FF}_C}. \quad (23)$$

2.7 Language modelling output projection

After the final decoder layer, we project each hidden representation from the model dimension C to the vocabulary dimension V . This produces one logit for every vocabulary item at each token position.

$$\text{Input shape} = \boxed{B \times T \times C}, \quad (24)$$

$$\text{Output shape} = \boxed{B \times T \times V}, \quad (25)$$

$$\text{FLOPs} = 2 \times B \times T \times C \times V \quad (26)$$

$$= \boxed{2BTCV}. \quad (27)$$

2.8 Total

The total amount of FLOPs in the prefill stage corresponds to:

$$\text{FLOPs} = BTC + 6LBTC^2 + 2LBCT^2 + 2LBCT^2 + 2LBT C^2 + 4LBT CFF_C + 2BTCV \quad (28)$$

$$= \boxed{BTC + 8LBTC^2 + 4LBCT^2 + 4LBT CFF_C + 2BTCV}. \quad (29)$$

3 Decoding Stage with KV cache

During the decoding stage, instead of passing T tokens, we only pass one new token together with a KV cache containing the previously computed keys and values. Here, $T_q = 1$ and $T = T_k$ denotes the total number of key-value entries available in the KV cache, including the current token. This simplifies many of the calculations.

3.1 Embedding / Positional Encodings

$$\text{Input shape} = \boxed{B \times 1} \quad (30)$$

$$\text{Output shape} = \boxed{B \times 1 \times C} \quad (31)$$

$$\text{FLOPs} = \boxed{BC} \quad (32)$$

3.2 Decoder: Q, K, V projections

As we have cached the K and V of the previously processed tokens we only need to do these projections for the single new token.

$$\text{Input shape} = \boxed{B \times 1 \times C}, \quad (33)$$

$$\text{Output shape} = \boxed{B \times H \times 1 \times H_d}, \quad (34)$$

$$\text{FLOPs} = L \times 3 \times 2 \times B \times 1 \times C \times C \quad (35)$$

$$= \boxed{6LBC^2}. \quad (36)$$

3.3 Decoder: Attention $QK^\top$

The benefit here is that we now only have one T_q token. This means FLOPs no longer scale quadratically with context length.

$$\text{Input shape} = \boxed{B \times H \times 1 \times H_d}, \quad (37)$$

$$\text{Output shape} = \boxed{B \times H \times 1 \times T_k}, \quad (38)$$

$$\text{FLOPs} = L \times 2 \times B \times H \times 1 \times H_d \times T_k \quad (39)$$

$$= \boxed{2LBCT}. \quad (40)$$

3.4 Decoder: Attention $\text{softmax}(QK^\top)V$

Also in the multiplication with the value projection it is a huge benefit that FLOPs don't scale quadratically with the context length anymore.

$$\text{Input shape} = \boxed{B \times H \times 1 \times T_k}, \quad (41)$$

$$\text{Output shape} = \boxed{B \times H \times 1 \times H_d}, \quad (42)$$

$$\text{FLOPs} = L \times 2 \times B \times H \times 1 \times T_k \times H_d \quad (43)$$

$$= \boxed{2LBCT}. \quad (44)$$

3.5 Decoder: Linear attention projection

As we only pass one token for each batch this also becomes more efficient.

$$\text{Input shape} = \boxed{B \times 1 \times C}, \quad (45)$$

$$\text{Output shape} = \boxed{B \times 1 \times C}, \quad (46)$$

$$\text{FLOPs} = L \times 2 \times B \times 1 \times C \times C \quad (47)$$

$$= \boxed{2LBC^2}. \quad (48)$$

3.6 Decoder: Feed forward

We also make sure the feed forward doesn't scale with context length anymore.

$$\text{Input shape} = \boxed{B \times 1 \times C}, \quad (49)$$

$$\text{Output shape} = \boxed{B \times 1 \times C}, \quad (50)$$

$$\text{FLOPs} = L(2 \times B \times 1 \times C \times \text{FF}_C + 2 \times B \times 1 \times \text{FF}_C \times C) \quad (51)$$

$$= \boxed{4LBC\text{FF}_C}. \quad (52)$$

3.7 Language modelling output projection

We also only predict the next token for the last token not for every token in the context.

$$\text{Input shape} = \boxed{B \times 1 \times C}, \quad (53)$$

$$\text{Output shape} = \boxed{B \times 1 \times V}, \quad (54)$$

$$\text{FLOPs} = 2 \times B \times 1 \times C \times V \quad (55)$$

$$= \boxed{2BCV}. \quad (56)$$

3.8 Total

The total amount of FLOPs in the decode stage with a KV-cache stage corresponds to:

$$\text{FLOPs} = BC + 6LBC^2 + 2LBCT + 2LBCT + 2LBC^2 + 4LBCFF_C + 2BCV \quad (57)$$

$$= \boxed{BC + 8LBC^2 + 4LBCT + 4LBCFF_C + 2BCV}. \quad (58)$$

To predict the next token we effectively made the flops scale linearly with the context length, T instead of quadratically. However it is important that this is the FLOPs to predict 1 new token. When predicting a sequence of new tokens, T itself also scales linearly as we continuously add the K, V of the new token to the KV cache which makes the FLOPs necessary to generate a sequence of length T still quadratic in terms of T.

Without KV Cache

With KV Cache

$$\begin{aligned} \text{FLOPs}_{\text{no cache}} &= BTC \\ &+ 8LBT C^2 \\ &+ 4LBCT^2 \\ &+ 4LBT CFF_C \\ &+ 2BTCV \end{aligned}$$

$$\begin{aligned} \text{FLOPs}_{\text{cache}} &= BC \\ &+ 8LBC^2 \\ &+ 4LBCT \\ &+ 4LBCFF_C \\ &+ 2BCV \end{aligned}$$

$$\text{Attention scaling: } \boxed{\mathcal{O}(T^2)}$$

$$\text{Attention scaling: } \boxed{\mathcal{O}(T)}$$

4 Memory

We estimate the amount of data read from memory. We use 32-bit floating-point values, so each parameter occupies 4 bytes. This estimate undercounts

total memory traffic because intermediate activations must also be written to and read from memory.

4.1 Prefill and Decode Without a KV Cache

Data read	Number of elements
Embeddings	$B \times T \times C$
Positional encodings	$T \times C$
Q, K, V projection weights	$3 \times L \times C \times C$
Attention output projection weights	$L \times C \times C$
Feed-forward weights	$2 \times L \times C \times \text{FF}_C$
Language modelling head weights	$C \times V$

Table 2: Estimated elements read during prefill or decode without a KV cache.

The total number of elements read is:

$$N_{\text{read}} = B \times T \times C + T \times C + 3 \times L \times C \times C \quad (59)$$

$$+ L \times C \times C + 2 \times L \times C \times \text{FF}_C + C \times V \quad (60)$$

$$= \boxed{BTC + TC + 4LC^2 + 2LC\text{FF}_C + CV}. \quad (61)$$

Because each element occupies 4 bytes:

$$M_{\text{read}} = 4 (BTC + TC + 4LC^2 + 2LC\text{FF}_C + CV) \text{ bytes.} \quad (62)$$

4.2 Additional KV-Cache Reads During Decode

During decoding with a KV cache, both the cached keys and cached values must be read:

$$N_{\text{KV read}} = 2 \times B \times L \times T \times H \times H_d \quad (63)$$

$$= \boxed{2BLTC}. \quad (64)$$

In bytes, this is:

$$M_{\text{KV read}} = 4 (2BLTC) \text{ bytes.} \quad (65)$$

5 Roofline Math

Arithmetic intensity is the number of floating-point operations performed per byte read from memory:

$$I = \frac{\text{FLOPs}}{M_{\text{read}}}. \quad (66)$$

5.1 Prefill

$$I_{\text{prefill}} = \frac{BTC + 8LBTC^2 + 4LBCT^2 + 4LBTCFF_C + 2BTCV}{4(BTC + TC + 4LC^2 + 2LCFF_C + CV)} \quad (67)$$

$$= \frac{BTC(1 + 8LC + 4LT + 4LFF_C + 2V)}{4C(T(B + 1) + 4LC + 2LFF_C + V)} \quad (68)$$

$$= \boxed{\frac{BT(1 + 8LC + 4LT + 4LFF_C + 2V)}{4(T(B + 1) + 4LC + 2LFF_C + V)}}. \quad (69)$$

5.2 Decode With a KV Cache

$$I_{\text{decode}} = \frac{BC + 8LBC^2 + 4LBCT + 4LBCFF_C + 2BCV}{4(BC + C + 4LC^2 + 2LCFF_C + CV + 2LBTC)} \quad (70)$$

$$= \frac{BC(1 + 8LC + 4LT + 4LFF_C + 2V)}{4C(B + 1 + 4LC + 2LFF_C + V + 2LBT)} \quad (71)$$

$$= \boxed{\frac{B(1 + 8LC + 4LT + 4LFF_C + 2V)}{4(B + 1 + 4LC + 2LFF_C + V + 2LBT)}}. \quad (72)$$

5.3 Arithmetic Intensity of My Transformer

I trained a transformer from scratch and used it for my KV-cache experiments. Its parameters are:

Parameter	Value
L	8
C	1024
FF_C	4096
T	4096
V	4096
B	1

Table 3: Parameters used for the arithmetic-intensity calculation.

Filling these parameters into the equations above gives:

$$I_{\text{prefill}} \approx 3109.94 \text{ FLOPs/byte}, \quad (73)$$

$$I_{\text{decode}} \approx 0.50 \text{ FLOPs/byte}. \quad (74)$$

Remember these values are an upper bound as my memory transfer estimates do not include writing and reading of intermediate activation results within the transformer.

We can compare these arithmetic intensities with the operations per byte supported by some well-known GPUs:

GPU	TFLOPS FP32 Tensor	Bandwidth (TB/s)	Ops/Byte
M1 Pro	4.6	0.20	23
RTX 5080	56.28	0.96	58.625
A100	312	1.50	208
H100	989	3.35	295.22
B200	2500	8.00	312.5
Rubin	2000	22.0	90.9

Table 4: GPU compute performance and memory bandwidth.

From this we can conclude that prefill stage is compute-bound, while the decode stage is memory-bound under this model. This means that using a GPU with more FP32 GFLOPs but the same memory bandwidth primarily benefits prefill. Improving decode performance instead requires greater memory bandwidth or reduced memory traffic.

5.4 Effect of Batch Size

Increasing the batch size allows the model weights to be reused across more tokens. With $B = 100$, the estimated decode arithmetic intensity increases to:

$$I_{\text{decode}, B=100} \approx 1.26 \text{ FLOPs/byte.} \quad (75)$$

The improvement remains limited because KV-cache traffic also scales with batch size. At the maximum context length, the FP32 KV cache alone would exceed 26 GB, excluding model weights and activation memory.

6 Conclusion

A KV cache makes the FLOPs of each decode step scale linearly instead of quadratic in terms of the current context length. This substantially reduces the computation required to predict the next token, but it also lowers arithmetic intensity. This makes prefill compute-bound while decode is memory-bound. This also shows that simply using a GPU with more GFLOPs doesn’t necessarily improve throughput.